

FreshMart

Online Grocery Delivery Platform

Software Requirements Specification

Version 1.0 | April 2026

.NET 10 | Angular 21 | Docker | Jenkins CI/CD

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the complete functional and non-functional requirements for **FreshMart**, an online grocery delivery platform built on a microservices architecture. It serves as the authoritative reference for developers, testers, and stakeholders.

1.2 Scope

FreshMart enables customers to browse a product catalogue, manage a shopping cart, place orders, pay online via Razorpay, and track deliveries in real time. The platform supports four user roles: **Customer**, **Admin**, **StoreManager**, and **DeliveryDriver**. It is composed of **13 independent microservices** communicating via REST APIs and RabbitMQ message queues, deployed as Docker containers with a Jenkins CI/CD pipeline.

1.3 Definitions and Acronyms

Term	Definition
JWT	JSON Web Token - stateless auth token (15 min expiry)
YARP	Yet Another Reverse Proxy - API Gateway library for .NET
RabbitMQ	Open-source message broker using AMQP protocol
SignalR	ASP.NET library for real-time WebSocket communication
EF Core	Entity Framework Core - ORM for .NET
CQRS	Command Query Responsibility Segregation pattern
OTP	One-Time Password for email verification / password reset
HMAC	Hash-based Message Authentication Code
BCrypt	Adaptive password hashing algorithm with built-in salt
DFD	Data Flow Diagram
ER	Entity Relationship Diagram

2 Overall Description

2.1 Product Perspective

FreshMart is a cloud-native, containerised web application. The Angular frontend communicates exclusively with the API Gateway (port 5000), which routes requests to the appropriate microservice. All services are deployed as Docker containers orchestrated via Docker Compose and built through a Jenkins CI/CD pipeline.

2.2 User Classes

Role	Description	Key Capabilities
Customer	Registered shopper	Browse, cart, order, pay, track, review, support chat
Admin	Platform administrator	Manage users, products, orders, coupons, refunds, analytics
StoreManager	Store operations manager	Manage catalogue, stock, discounts, view orders
DeliveryDriver	Delivery personnel	View assigned deliveries, update status, mark delivered

2.3 Operating Environment

- Backend: .NET 10 ASP.NET Core microservices in Docker containers
- Frontend: Angular 21 with SSR, served via Nginx on port 4200
- Database: Microsoft SQL Server 2022 (separate DB per service)
- Cache: Redis 7 (cart storage, product caching)
- Message Broker: RabbitMQ 3.13 with management UI on port 15672
- CI/CD: Jenkins running in Docker with .NET 10 and Docker CLI
- OS: Linux containers (Alpine/Debian) on Windows Docker Desktop host

3 Functional Requirements

3.1 Authentication Service

ID	Requirement	Priority
FR-A01	Register with email, password, first/last name	High
FR-A02	Hash passwords using BCrypt cost factor 11	High
FR-A03	Issue JWT (15 min) + refresh token (30 days) on login	High
FR-A04	Token rotation on every refresh request	High
FR-A05	Google OAuth Sign-In and account linking	Medium
FR-A06	Send 6-digit OTP via email for verification and password reset	High
FR-A07	Hash OTP with BCrypt, expire after 10 minutes	High
FR-A08	Prevent email enumeration with identical error messages	High
FR-A09	Save multiple delivery addresses per user	Medium
FR-A10	Revoke refresh token immediately on logout	High

3.2 Product Service

ID	Requirement	Priority
FR-P01	Full-text search across product name and description	High
FR-P02	Filter by category, price range, and brand	High
FR-P03	Pagination (page + pageSize) on all list endpoints	High
FR-P04	Cache product search results in Redis with TTL	Medium
FR-P05	Percentage discounts on individual products	High
FR-P06	Low-stock alerts for Admin/StoreManager	Medium
FR-P07	Atomically deduct stock on order placement	High
FR-P08	Seed 50+ products and categories on first startup	Low

3.3 Order Service

ID	Requirement	Priority
FR-O01	Create orders with itemised line items and delivery address	High
FR-O02	Order state machine: Pending to PaymentConfirmed to Processing to Shipped to Delivered	High
FR-O03	Prevent cancellation of shipped or delivered orders	High

ID	Requirement	Priority
FR-O04	Publish OrderCreatedEvent to RabbitMQ on order creation	High
FR-O05	Proxy Razorpay payment creation and verification	High
FR-O06	Driver dashboard with delivery statistics	Medium

3.4 Payment Service

ID	Requirement	Priority
FR-PAY01	Create Razorpay orders and return order_id + public key to frontend	High
FR-PAY02	Verify HMAC-SHA256 signature on payment callback	High
FR-PAY03	Receive and verify Razorpay server-to-server webhooks	High
FR-PAY04	Support full refunds via Razorpay API (Admin only)	Medium
FR-PAY05	Never store card/UPI details - all handled by Razorpay	High

3.5 Notification Service

ID	Requirement	Priority
FR-N01	Send welcome email on user registration via RabbitMQ	High
FR-N02	Send OTP emails for verification and password reset	High
FR-N03	Send order confirmation email with itemised table	High
FR-N04	Send delivery invoice email on order delivery	High
FR-N05	Push real-time notifications to browser via SignalR	Medium
FR-N06	Persist notifications to DB for inbox history	Medium
FR-N07	Support mark-as-read and delete for notifications	Low

4 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords hashed with BCrypt cost factor 11
NFR-02	Security	JWT access token expires in 15 minutes
NFR-03	Security	Razorpay KeySecret never sent to frontend
NFR-04	Security	Global exception middleware prevents stack trace leakage
NFR-05	Security	FluentValidation pipeline rejects malformed input
NFR-06	Performance	Redis caching for product search results
NFR-07	Performance	Cart stored in Redis (O(1) read/write)
NFR-08	Reliability	RabbitMQ messages are durable - survive broker restart
NFR-09	Reliability	NotificationRelay has HTTP fallback if RabbitMQ unavailable
NFR-10	Reliability	RabbitMQ consumer retries with exponential backoff (10 attempts)
NFR-11	Scalability	Each microservice independently deployable and scalable
NFR-12	Availability	Health check /health endpoint on every service
NFR-13	Observability	Serilog structured logging with request logging middleware
NFR-14	Maintainability	CQRS pattern separates read and write operations
NFR-15	Maintainability	Result<T> pattern replaces exceptions for business failures

5 System Architecture Diagram

The diagram below shows the complete microservices architecture of FreshMart. The Angular frontend communicates only with the API Gateway. The gateway routes requests to the appropriate service. Services communicate asynchronously via RabbitMQ (dashed lines) and synchronously via HTTP (solid arrows). Three background zones show the Client/Gateway layer, Microservices layer, and Infrastructure layer.

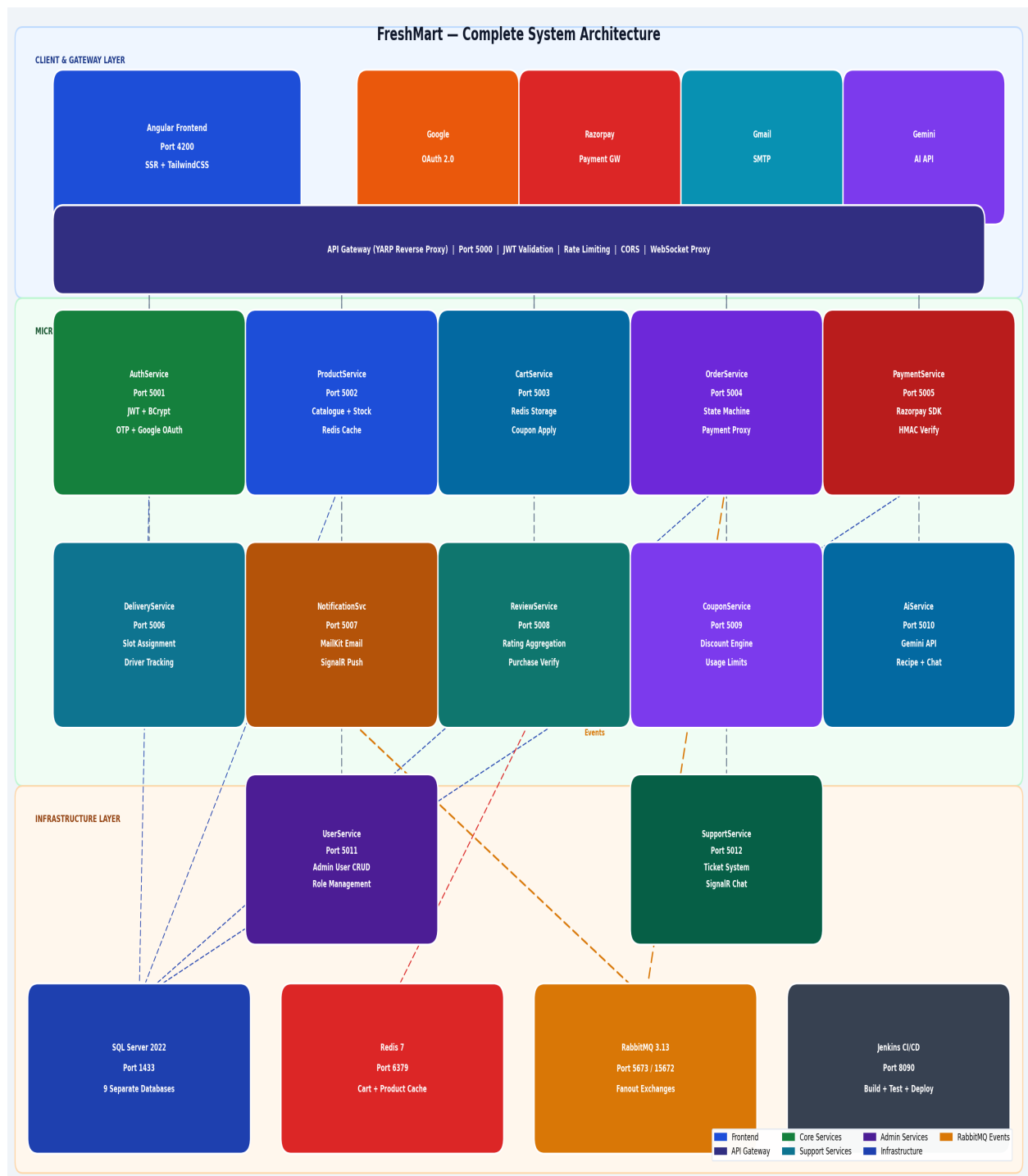


Figure 1: FreshMart Complete System Architecture - 13 microservices, API Gateway, Redis, RabbitMQ, SQL Server

6 Entity Relationship Diagram

The ER diagram shows all major entities across the FreshMart platform. Each microservice owns its own database. Yellow rows indicate Primary Keys (PK), blue rows indicate Foreign Keys (FK). Arrows show 1:N relationships between entities.

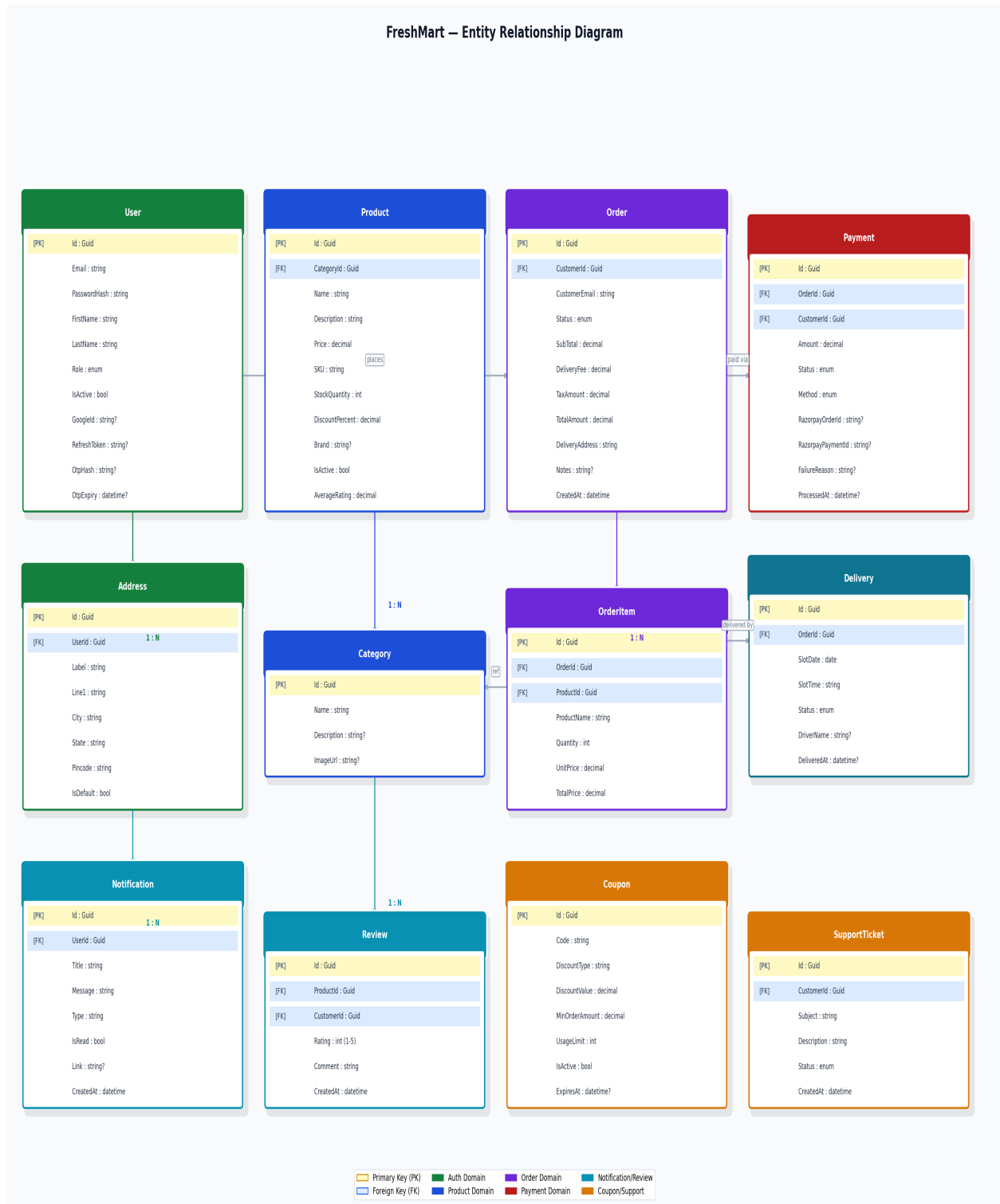


Figure 2: FreshMart Entity Relationship Diagram - 11 entities across 9 databases

7 Data Flow Diagram (Level 1)

The DFD shows how data flows between external entities, processes, and data stores. Rectangles = external entities. Circles = processes. Filled bars = data stores. Solid arrows = synchronous HTTP flows. Dashed arrows = async/database flows.

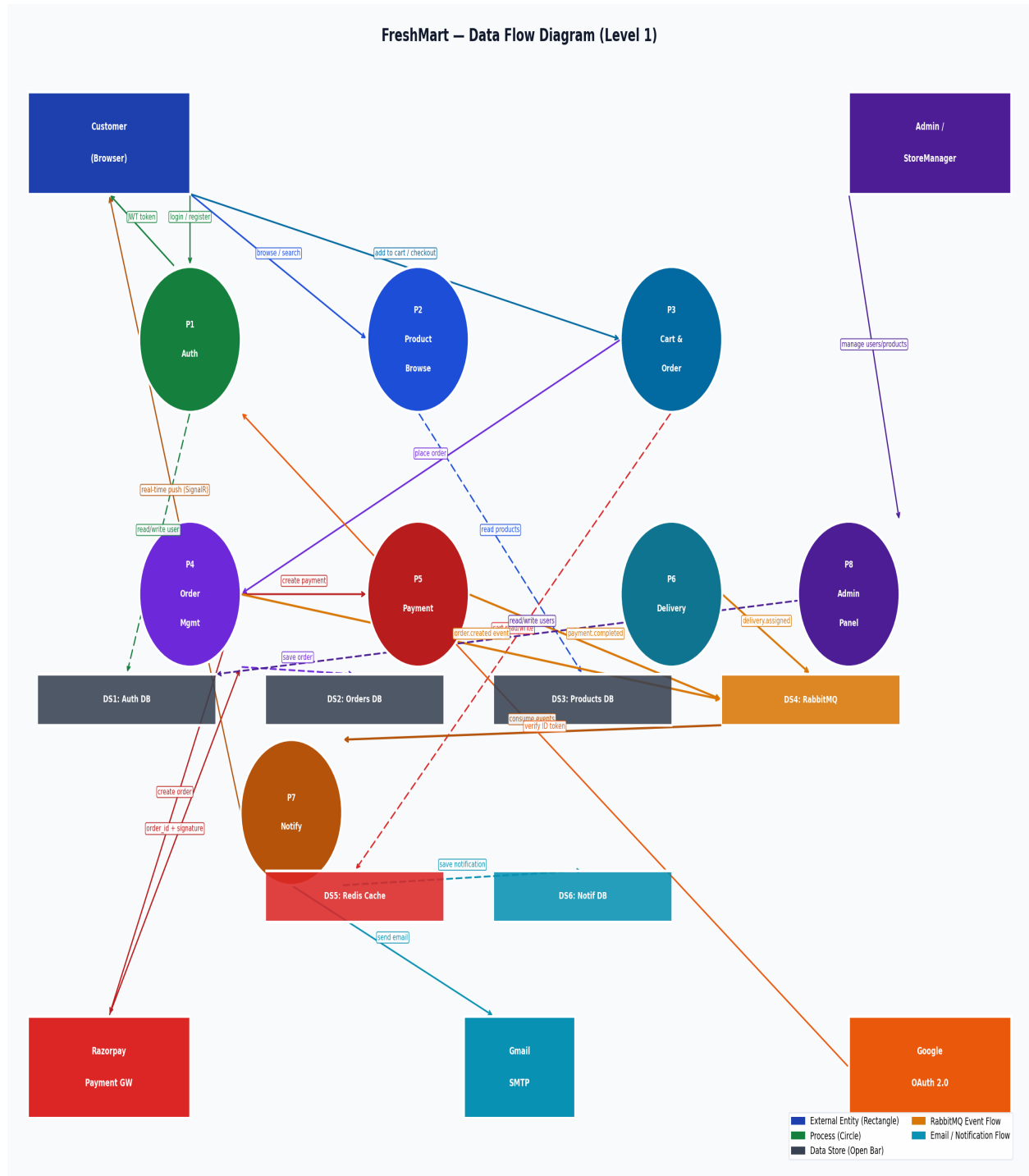


Figure 3: FreshMart DFD Level 1 - 8 processes, 6 data stores, 5 external entities

8 Payment Flow Sequence Diagram

The sequence diagram shows the complete Razorpay payment flow across 6 actors: Customer Browser, API Gateway, OrderService, PaymentService, Razorpay API, and NotificationService. The flow covers order creation, payment, HMAC signature verification, and notification dispatch.

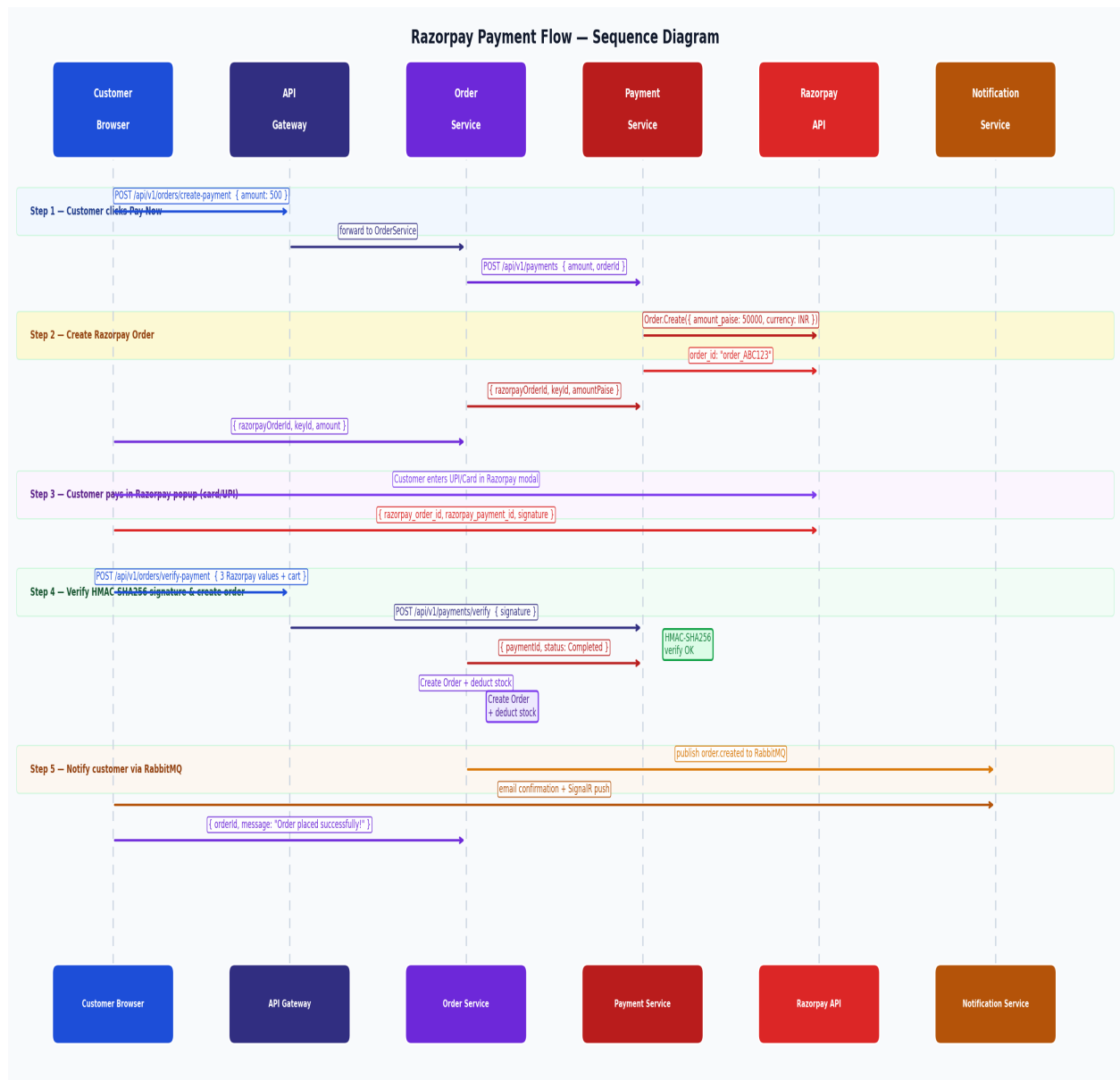


Figure 4: Razorpay Payment Flow Sequence Diagram - 5 steps across 6 actors

9 Microservice Internal Architecture

Each microservice follows Clean Architecture with 4 layers: **API Layer** (controllers, HTTP), **Application Layer** (handlers, CQRS, business logic), **Domain Layer** (entities, aggregates, interfaces), and **Infrastructure Layer** (EF Core, external services, messaging). Arrows show bidirectional dependency between layers.



Figure 5: Clean Architecture pattern applied to 6 core microservices

10 Service Catalogue

Service	Port	Database	Technology	Key Responsibility
ApiGateway	5000	None	YARP	Reverse proxy, JWT validation, rate limiting
AuthService	5001	GroceryAuth	.NET + EF Core	Login, register, JWT, OTP, Google OAuth
ProductService	5002	GroceryProducts	.NET + Redis	Product catalogue, stock, categories
CartService	5003	Redis only	.NET + Redis	Shopping cart, coupon application
OrderService	5004	GroceryOrders	.NET + EF Core	Order lifecycle, payment proxy
PaymentService	5005	GroceryPayments	.NET + Razorpay	Razorpay integration, signature verify
DeliveryService	5006	GroceryDelivery	.NET + EF Core	Delivery slots, driver tracking
NotificationService	5007	GroceryNotifications	.NET + MailKit	Email, SignalR push, in-app notifications
ReviewService	5008	GroceryReviews	.NET + EF Core	Product reviews, rating aggregation
CouponService	5009	GroceryCoupons	.NET + EF Core	Coupon validation and discount calculation
AiService	5010	None	.NET + Gemini	AI chat and recipe suggestions
UserService	5011	GroceryAuth (shared)	.NET + EF Core	Admin user management
SupportService	5012	GrocerySupport	.NET + SignalR	Customer support tickets and live chat

11 RabbitMQ Integration Event Map

Exchange (Topic)	Publisher	Subscriber	Trigger / Payload
user.registered	AuthService	NotificationService	New user signs up - sends welcome email
otp.requested	AuthService	NotificationService	OTP requested - sends OTP email
order.created	OrderService	NotificationService	Order placed - sends confirmation email
order.status-changed	OrderService	NotificationService	Status updated - sends status email + push
payment.completed	PaymentService	OrderService	Payment verified - confirms order
payment.failed	PaymentService	OrderService	Payment failed - cancels order
inventory.updated	ProductService	Admin alerts	Stock changed - low stock alert
delivery.assigned	DeliveryService	NotificationService	Slot assigned - sends delivery email
order.cancelled	OrderService	NotificationService	Order cancelled - sends cancellation email

12 Security Design

12.1 Authentication and Token Security

- Passwords hashed with BCrypt cost factor 11 - never stored in plain text
- JWT access token expires in 15 minutes - limits exposure window if stolen
- Refresh token is 64 cryptographically random bytes (Base64) stored in DB
- Token rotation: every refresh issues a new refresh token, old one invalidated
- Deactivated users blocked at both login and token refresh endpoints
- Refresh token immediately nulled on user deactivation or deletion
- ClockSkew = TimeSpan.Zero - no grace period on JWT expiry

12.2 Payment Security

- Razorpay KeySecret never sent to frontend - only KeyId (public) is sent
- Payment signature: HMAC-SHA256(razorpayOrderId|razorpayPaymentId, KeySecret)
- Webhook signature: HMAC-SHA256(rawJsonBody, WebhookSecret)
- Card/UPI details never touch FreshMart servers - handled entirely by Razorpay
- Refund endpoint restricted to Admin role only

12.3 API Security

- All endpoints protected by JWT Bearer authentication
- Role-based authorization: [Authorize(Roles = Admin)] on admin endpoints
- FluentValidation pipeline rejects malformed input before handler runs
- GlobalExceptionHandler prevents stack trace leakage on unhandled errors
- CORS restricted to frontend origin and API Gateway only
- Rate limiting configured in API Gateway via YARP middleware

13 CI/CD Pipeline

Stage	Description	Tool
1. Checkout	Clean clone from GitHub main branch	Jenkins + Git
2. Inject Secrets	Copy .env from Jenkins credentials store	Jenkins Credentials
3. Build .NET	dotnet restore + dotnet build --configuration Release	.NET 10 CLI
4. Test .NET	Run 7 test projects (100+ NUnit tests), publish JUnit	dotnet test
5. Docker Build	14 images built in parallel with --build-arg for secrets	Docker
6. Deploy	docker compose up -d --no-build --pull never	Docker Compose

The Jenkins pipeline runs on a Docker container with .NET 10, Node 20, and Docker CLI. All secrets (JWT secret, RabbitMQ credentials, Razorpay keys, Gmail password) are stored as a single Jenkins Secret File credential and injected at build time. The Razorpay public key is baked into the Angular bundle via Docker --build-arg. Secrets are deleted from the workspace in the post always cleanup block.

14 Technology Stack

Layer	Technology	Version	Purpose
Backend	.NET / ASP.NET Core	10.0	All 13 microservices
Backend	Entity Framework Core	10.0	ORM for SQL Server
Backend	MediatR	12.2	CQRS command/query dispatch
Backend	FluentValidation	11.9	Input validation pipeline
Backend	Serilog	10.0	Structured logging
Backend	MailKit	4.7	SMTP email sending
Backend	Razorpay SDK	2.1	Payment gateway integration
Backend	BCrypt.Net	4.0	Password hashing
Backend	YARP	2.x	Reverse proxy / API Gateway
Frontend	Angular	21.2	SPA framework with SSR
Frontend	TailwindCSS	3.x	Utility-first CSS framework
Frontend	TypeScript	5.x	Type-safe JavaScript
Database	SQL Server	2022	Relational data storage
Cache	Redis	7	Cart and product caching
Messaging	RabbitMQ	3.13	Async event bus
Container	Docker	24+	Service containerisation
CI/CD	Jenkins	2.x	Build, test, deploy pipeline
Testing	NUnit + NSubstitute	4.x	100+ unit tests, 7 projects